Names: Christian Ezekiel L. Carvajal & John Miko P. Sarsalijo	**Subject: BSCS 3112 - AI Lab (Lesson 6: State Machines)**
Course/Year: BSCS - DS 3rd year	**Date: October 11, 2026**

LAB ACTIVITY 6.1: STATEFUL RESTAURANT ORDER AGENT & DYNAMIC TOOL CALLING ENGINE

1. Business Context & Problem Formulation

Perpetual Bistro operates as a high-volume campus dining facility serving students, faculty, and administrative staff at the University of Perpetual Help System DALTA. During peak lunch rush hours, traditional counter ordering suffers from queue congestion, order entry errors, and delays in communicating dietary restrictions to the kitchen. To modernize this interaction, the bistro initiated a digital transformation project prototyping an automated table-side ordering terminal.

Standard conversational AI models struggle in food-service environments when deployed through stateless single-turn API wrappers. Without persistent memory, an LLM treats each customer utterance in isolation, dropping previously ordered items when a guest asks an unrelated question about ingredients. Even worse, generative models frequently hallucinate imaginary dishes, fabricate incorrect prices, or sell meals that the kitchen ran out of thirty minutes earlier. To overcome these hazards, our engineering team implemented a zero-cloud conversational state machine that binds a local language model (qwen3.5:4b running inside Ollama) to a deterministic SQLite relational database.

2. Conversational Finite State Machine (FSM) Architecture

The dialog engine coordinates customer interactions through an explicit Finite State Machine (FSM). Rather than letting the model freely guide conversation, the

Python controller maintains an authoritative state variable synchronized with the table_sessions database table. The interaction moves through five distinct dialogue states:

- STATE_GREETING: Initializes the session, welcomes the diner to Perpetual Bistro, captures or confirms their physical table number, and prompts them for their initial dining request.
- STATE_MENU_BROWSING: Activates when customers ask about available offerings, category lists (Burgers, Pastas, Sides, Beverages, Desserts), item descriptions, pricing, or allergen profiles.
- STATE_TAKING_ORDER: Engages when the customer selects items. The system routes choices to the add_item_to_order tool, recording specific customization instructions such as 'extra cheese, no onions'.
- STATE_REVIEW: Triggers when the guest requests their bill, checks their tab, or prepares to pay. The agent presents an itemized summary containing line items, special instructions, subtotal, 12% VAT, and grand total.
- STATE_CONFIRMED: Represents the terminal state reached when the customer verifies their order. The engine executes atomic inventory deductions in SQLite, outputs an immutable digital receipt ticket, and closes the session.

State transitions follow rigorous bidirectional guardrails. If a customer in STATE_REVIEW decides to add another beverage or remove an unwanted side dish, the state controller transitions the session back into STATE_TAKING_ORDER, updates the cart, and returns to STATE_REVIEW only when the updated total is ready. This design prevents unintended premature checkouts while giving guests total control over their tab.

3. Relational Database Modeling & Inventory Integrity

Data persistence relies on SQLite (restaurant.db), initialized through init_restaurant_db.sql. The schema divides operational records across three normalized tables:

- menu_items: Stores master catalog items with columns (id, name, category, price, stock_qty, allergens, description). This serves as the single source of truth for stock levels and dietary warnings.

- table_sessions: Tracks session lifecycles with columns (session_id, table_no, status, created_at, updated_at), recording the active FSM state.
- order_line_items: Contains active cart entries with columns (id, session_id, item_id, quantity, special_instructions, unit_price), linked by foreign keys to prevent orphan records.

To test inventory safeguards, the menu includes an intentionally depleted item: 'Crispy Buffalo Chicken Tenders' with stock_qty set to 0. When our test script attempted to order this dish, the tool execution logic intercepted the request, aborted the insertion, and queried available alternatives in the same category. The language model then informed the guest of the shortage and suggested alternative chicken and burger meals.

4. Dynamic Function Calling & Local LLM Tool Binding

The interface between natural language customer dialogue and database operations uses Ollama's native tool calling protocol. In fsm_agent.py, we defined JSON function schemas matching Python routines in db_tools.py. These include get_menu_by_category, check_item_allergen, check_availability, add_item_to_order, remove_item_from_order, update_item_quantity, view_current_order, and confirm_order_and_deduct_stock.

During execution, Ollama parses user intent and outputs structured function calls instead of raw conversational text. The Python controller captures these tool calls, executes the corresponding SQLite queries, and returns the result dictionary wrapped in a message with role 'tool'. The model reads this structured feedback and formulates a courteous natural language reply. During extended multi-turn testing, we identified that Qwen's internal chain-of-thought tokens consumed the default 4,096 token buffer. Expanding num_ctx to 8,192 in the request options completely eliminated token truncation.

5. Conversational Guardrails, Dietary Safety & Transaction Integrity

Real-world dining deployments require defensive boundaries to prevent system crashes and health risks:

- Out-of-Domain Deflection: System prompts strictly restrict BistroBot to dining assistance. Inquiries concerning non-restaurant topics receive polite refusals redirecting customers back to the bistro catalog.
- Allergen Verification: When customers report lactose intolerance or dietary concerns, check_item_allergen directly checks the database allergen tags. In our verification run, the agent accurately alerted the customer that the Perpetual Classic Burger contains dairy.
- Customization Parsing: Customer modifiers like 'no onions, extra cheese' are parsed directly into the special_instructions column, guaranteeing kitchen staff receive exact preparation details.
- ACID Transactional Checkout: confirm_order_and_deduct_stock executes inside an explicit SQLite BEGIN TRANSACTION block. If inventory levels change mid-order or if any stock check fails, the transaction issues a ROLLBACK, ensuring inventory counts never go negative.

6. Voice I/O Decoupling & Multimodal Integration Readiness

In accordance with Lesson 6 Subtopic 6.5, we decoupled agent reasoning from user input channels. The IOChannel class exposes abstract get_input() and send_output() methods. In terminal mode, get_input() reads console input via the rich library, while send_output() prints color-coded panels. In the web dashboard, get_input() connects to the browser's Web Speech API for voice recognition, while send_output() feeds speech synthesis. This decoupling ensures that switching between text and speech requires zero modifications to the core finite state machine.

7. Interface Implementations: Terminal TUI & Web Tablet Dashboard

Our deliverable includes two complete user interface implementations:

- Rich Terminal TUI (virtual_waiter_prototype.py): Displays table status badges, active FSM state pills, live cart tables, dialogue speech bubbles, and digital receipts entirely within the console.
- Web Tablet Dashboard (web_dashboard.py): Implements Option 1 from the professor's lab brief. Running on localhost:8000 via FastAPI, it delivers a table-side tablet experience featuring a Left Panel conversation stream with speech recognition, a Right Panel real-time cart with manual +/- quantity buttons, and a Bottom Bar checkout trigger that generates a receipt modal.

8. Tandem Pair Programming Reflection & Role Analysis

The project strictly practiced Agile pair programming, rotating roles between Driver and Navigator across every development milestone. The Driver operated the keyboard, writing SQL queries, constructing Python classes, and running tests. Meanwhile, the Navigator audited code against the rubric checklist, identified edge cases, verified state transitions, and supervised token context efficiency.

During Milestone 1, Christian Ezekiel L. Carvajal acted as Driver to author the database schema and tool bindings, while John Miko P. Sarsalijo navigated table normalization, quantity grouping in add_item_to_order, and 12% VAT calculations. Halfway through the lab during Milestone 2, roles rotated: Sarsalijo drove the implementation of the 5-state FSM and Ollama tool schemas in fsm_agent.py, while Carvajal navigated context window expansion (raising num_ctx to 8192) and fallback text synthesis. In Milestone 3, Carvajal drove the FastAPI web tablet dashboard and Rich TUI, while Sarsalijo navigated live SQLite synchronization and ACID stock decrement rollbacks. Finally, in Milestone 4, Sarsalijo drove the local Edge-TTS voice audio pipeline and phonetic Taglish sanitization, while Carvajal navigated emoji stripping and Web Speech API mic integration. This equal 50-50 rotation eliminated single-developer fatigue, guaranteed continuous code review, and delivered clean, defect-free code.

9. Verification Evidence & Multi-Turn Test Execution

We tested the full system using test_fsm_agent.py, executing a continuous 7-turn conversation against local Ollama qwen3.5:4b. All turns succeeded with zero errors:

- Turn 1 (Menu Browsing): Customer inquired about burgers. Agent executed get_menu_by_category('Burgers') and listed available options while noting the depleted tenders.
- Turn 2 (Allergen Check): Customer queried dairy content. Agent executed check_item_allergen('Perpetual Classic Burger', 'Dairy') and accurately flagged the presence of dairy.
- Turn 3 (Order Placement): Customer ordered a burger (extra cheese, no onions) and Loaded Cheese Fries. Agent called add_item_to_order twice, establishing a subtotal of ₱260.00.
- Turn 4 (Out-of-Stock Handling): Customer requested Crispy Buffalo Chicken Tenders. Agent executed check_availability, detected 0 stock, and politely recommended in-stock items.
- Turn 5 (Item Substitution): Customer ordered an Iced Caramel Macchiato. Agent called add_item_to_order, updating the subtotal to ₱355.00 across 3 items.
- Turn 6 (Tab Review): Customer asked to see their tab. Agent called view_current_order, transitioned to STATE_REVIEW, and presented an itemized breakdown including 12% VAT.
- Turn 7 (Checkout & Confirmation): Customer confirmed their order. Agent called confirm_order_and_deduct_stock, transitioned to STATE_CONFIRMED, committed changes to SQLite, decremented inventory, and generated receipt #PB-3-7CC763.

10. Conclusion

Lab Activity 6.1 successfully demonstrates that pairing a local, lightweight language model (qwen3.5:4b) with a deterministic Finite State Machine and relational SQLite database solves the core challenges of automated ordering. By constraining dialogue transitions, binding queries to database tools, and decoupling I/O channels,

Perpetual Bistro gains a reliable, zero-cloud ordering assistant that prevents hallucinations and protects inventory integrity.